

National University of Computer and Emerging Sciences



Lab Manual 10 **DATA ANALYSIS AND VISUALIZATION Lab**

Course Instructor	
Lab Instructor (s)	Gullsher Chaudhary Rida Mahmood
Section	
Semester	Fall 2023

Department of Computer Science
FAST-NU, Lahore, Pakistan

1 Online R Interpreter

We will be working on [Google Collab](#) to run R programs.

Colaboratory, or "Colab" for short, allows you to write and execute R in your browser with "R" configuration. To do that we need to create a Colab notebook.

use %%R at the start of each block of R code.

.....WHY R.....

R is a popular programming language used for statistical computing and graphical presentation. Its most common use is to analyze and visualize data.

- It provides many statistical techniques (such as statistical tests, classification, clustering and data reduction)
- It is easy to draw graphs in R, like pie charts, histograms, box plot, scatter plot, etc++
- It works on different platforms (Windows, Mac, Linux)
- It is open-source and free
- It has a large community support
- It has many packages (libraries of functions) that can be used to solve different problems

2 Data Types

The following section describes the standard types that are built into the R interpreter. These datatypes are divided into different categories like numeric, sequences, mapping etc. Typecasting is also discussed below.

2.1 Built-in Types

The following chart summarizes the standard data types that are built into the R interpreter.

Sr#	Categories	Data Type	Examples
1	Numeric Types	int	-2, -1, 0, 1, 2, 3, 4, 5, int(20)
2		float	-1.25, -1.0, --0.5, 0.0, 0.5, 1.0, 1.25, float(20.5)
3		complex	1i, complex(i) here 'i' is complex
4	Character Type	str	'a', 'Hello!', str("Hello World")
5	Logical Type	bool	True, False, bool(5)

R has no command for declaring a variable for any datatype. A variable is created the moment you first assign a value to it. Variable names are case-sensitive. Just like in other languages, R allows you to assign values to multiple variables in one line.

```
# numeric
x <- 10.5
class(x)

# integer
x <- 1000L
class(x)

# complex
x <- 9i + 3
class(x)

# character/string
x <- "R is exciting"
class(x)

# logical/boolean
x <- TRUE
class(x)
```

2.2 Typecasting

The process of explicitly converting the value of one data type (int, str, float, etc.) to another data type is called type casting. Inbuilt functions 'as.typeName' shall be used for typecasting. as.numeric(value) can take a float or string literal as argument and returns a value of class 'int' type. In Type Casting, loss of data may occur as we enforce the object to a specific data type.

```
# cast to int
age <- "20"
flag <- False
as.numeric(age)
as.numeric(flag)
print(age)    // 20
print(flag)   // 0

# cast to str
age <- 20
flag <- 0
as.character(age)
as.character(flag)
print(age)    // "20"
print(flag)   // "False"
```

```
# cast to boolean
flagOne <- 1
flagTwo <- 0
as.logical(flagOne)
as.logical(flagTwo)
print(flagOne)      // True
print(flagTwo)      // False

typeof()
```

Notice the `typeof()` function used in the above example. Find out what it does. Execute the given example in Jupyter notebook to observe the result of type casting.

3 Operators

3.1 Math Operators

From **Highest** to **Lowest** precedence:

Operators	Operation	Example
<code>^</code>	Exponent	<code>2 ^ 3 = 8</code>
<code>%%</code>	Modulus/Remainder	<code>22 %% 8 = 6</code>
<code>%/%</code>	Integer division	<code>22 %/% 8 = 2</code>
<code>/</code>	Division	<code>22 / 8 = 2.75</code>
<code>*</code>	Multiplication	<code>3 * 3 = 9</code>
<code>-</code>	Subtraction	<code>5 - 2 = 3</code>
<code>+</code>	Addition	<code>2 + 2 = 4</code>

3.2 Comparison Operators

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to

Operator	Meaning
<	Less than
>	Greater Than
<=	Less than or Equal to
>=	Greater than or Equal to

3.3 Boolean Operators

There are three Boolean operators: &, &&, |, || and !

The && Operator's *Truth* Table:

Expression	Evaluates to
True && True	True
True && False	False
False && True	False
False && False	False

The || Operator's *Truth* Table:

Expression	Evaluates to
True True	True
True False	True
False True	True
False False	False

The ! Operator's *Truth* Table:

Expression	Evaluates to
! True	False
! False	True

4 If-else Conditions

R supports conditional statements i.e. `if`, `else if`, `else`. Comparison operators and Boolean operators written in the previous section can be used in if-elif-else statements.

R uses curly-brackets to define the scope in the code.

4.1 if Statement Example:

```
name = 'Alice'
if (name == 'Alice') {
  print('Hi, Alice.')
}
```

4.2 if-else Statement Example:

```
name = 'Bob'
if (name == 'Alice') {
  print('Hi, Alice.')
}
else {
  print('Hello, stranger.')
}
```

4.3 if, else if, else Statement Example:

```
name = 'Bob'
age = 30
if (name == 'Alice') {
  print('Hi, Alice.')
}
else if (age < 12) {
  print('You are not Alice, kiddo.')
}
else {
  print('You are neither Alice nor a little kid.')
}
```

R conditional statements only require `if` statement to execute. Both `else if` and `else` are optional and used as per requirement.

5 Loops

R has two types of loops i.e. `while`, `for`. Use Jupyter notebook to execute all code snippets given in the examples below to observe their results.

5.1 While Loop Example with break Statement

With the `while` loop we can execute a set of statements as long as a condition is true or the loop execution reaches a `break` statement.

```
while (True) {  
  print('Please type your name.')  name = readline()  
  if (name == 'your name') {  
    break  
  }  
}  
print('Thank you!')
```

`readline()` in the above example is a built-in R function which is discussed in the functions section below.

5.2 While Loop Example with next Statement

When the program reaches a `next` statement, the program execution immediately jumps back to the start of the loop.

```
while (True) {  
  print('Who are you?')  
  name = readline()  
  if (name != 'Joe') {  
    next  
  }  
  print('Hello, Joe. What is the password? (It is a fish.)')  
  if (name == 'swordfish') {  
    break  
  }  
}  
print('Access granted.')
```

5.3 for Loop Example with static value

```
print('My name is')  
for (i in 1:5) {  
  print(i)  
}
```

5.4 for Loop Example with list arguments

The `list` function can also be called with three arguments. The first two arguments will be the start and stop values, and the third will be the step argument. The step is the amount that the variable is increased by after each iteration.

```
for (i in list) {  
  print(i)  
}
```

6 Functions

6.1 Custom Functions

6.1.1 Simple Function Example

A function in R starts with *function* keyword followed by the function name with round brackets. Function parameters can be passed depending on the requirement.

```
my_function <- function(name) {  
  print(name)  
}  
my_function('Alice')  # Alice  
my_function('Bob')   # Bob
```

6.1.2 Function Example with Return Statement

A return statement consists of the following:

- The return keyword.
- The value or expression that the function should return.

```
my_func <- function(answerNumber) {  
  if (answerNumber < 0) {  
    return 'negative'  
  }  
  return 'positive'  
}  
r <- -1  
fortune <- my_func(r)  
print(fortune)
```

6.2 Built-in Functions

The R interpreter has a number of functions built into it that are always available. We have already covered a few built-in functions in the datatypes section above. Refer to [this link](#) for the complete list of R built-in functions.

6.2.1 Built-in Function Examples

Execute the code given below in your Jupyter notebook to find the results of built-in functions.

```
# abs integer number  
num <- -5  
print('Absolute value of -5 is:', abs(num))  
# Notice print here, it is also a built-in function  
  
# abs floating number  
fnum <- -1.45
```



```
print('Absolute value of 1.45 is:', abs(fnum))
```

```
# input function
```

```
x = readline()
```

```
print(x)
```

```
# max function
```

```
number <- c(3, 2, 8, 5, 10, 6)
```

```
largest_number = max(number);
```

```
print(largest_number)
```

```
# sum function
```

```
my_list <- c(1,3,5,2,4)
```

```
print(sum(my_list))
```

7 Preprocessing with R

```
#installing packages eg : ggplot2 , dplyr ,tidyr
```

```
library(tidyverse)
```

```
# importing files eg: data.csv ,images.zip,config.txt
```

```
list.files(path = "../input")
```

```
install.packages("psych")
```

```
library(psych)
```

```
# Reading data set
```

```
data <- read.csv("/kaggle/input/students-exam-scores/"  
                "Expanded_data_with_more_features.csv")
```

```
# Exploring the dimensions of our dataset
```

```
dim(data)
```

```
# displaying the column names
```

```
variable_names <- names(data)
```

```
print(variable_names)
```

8 Exercise (25 Marks)

Complete all tasks given below

8.1 Power function (3 Marks)

Create a function that takes a base number (b) and an exponent number (e) and returns the b^e calculation, without using built-in power function. Call the function in the next cell and show some outputs.

For Example:

myPower(2,3) return 8

8.2 Array Manipulation (7 Marks)

Write a function named "subtotal" takes as its arguments the following:

(1) an array of values;

The function should replace the contents of each cell with the sum of the contents of all the cells in the original array from the left end to the cell in question. Thus, for example, if the array passed to the function looks like this:

0	1	2	3	4
5.8	2.6	9.1	3.4	7.0

when the function returns, the array will have been changed so that it looks like this:

0	1	2	3	4
5.8	8.4	17.5	20.9	27.9

because $5.8 + 2.6 = 8.4$ and $5.8 + 2.6 + 9.1 = 17.5$ and so on. Note that the contents of cell 0 are not changed. The function should return the average of elements of original array.

8.3 Counter (7.5 Marks)

Write a function "**Counter**" that receives a character array and calculates Total characters, uppercases, lowercases and spaces then return them all. (In this question, all characters except spaces are counted as letters).

Sample Run:

Input:

Hello, I am a student of FAST University.

Values returned:

No. of letters are: 34

No. of spaces are: 7

No. of uppercase letters are: 7

No. of lowercase letters: 25

8.4 Median (7.5 Marks)

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`

Output: 2.00000

Explanation: merged array = `[1,2,3]` and median is 2.

Example 2:

Input: `nums1 = [1,2]`, `nums2 = [3,4]`

Output: 2.50000

Explanation: merged array = `[1,2,3,4]` and median is $(2 + 3) / 2 = 2.5$.

Submission Instructions

Always read the submission instructions carefully.

- Rename your Jupyter notebook to your roll number and download the notebook as **.ipynb** extension.
- To download the required file, go to **File->Download .ipynb**
- Only submit the **.ipynb** file. DO NOT **zip** or **rar** your submission file
- Submit this file on Google Classroom under the relevant assignment.
- Late submissions will **NOT AT ALL** be accepted